**Overview**

In brief, thus far, I have made TOAST-style pointing offset and pointing jitter operators which I have been able to run through simple (non-LAT specific) TOAST simulations. From these I can run a basic filter bin or ml mapmaker and can confirm that a simulated point source gets 'messed up'. I can also confirm that the operators adjust my pointing in the way I expect and I can run some basic beam statistics analysis on the output map (although the maps aren't themselves realistic so this was just for demonstration purposes) I have added these operators to a local dev version of TOAST and SOTODLIB for running with the toast_so_sim cli so that I can model a more realistic LAT camera. This is sort of where I have hit a little wall as I have struggled to figure out how/where exactly to add the execution of my operator in the sim workflow. My toast_so_sim appears to run my custom operator and apply the pointing error, but the output maps are largely identical to maps with no pointing errors. I presume this is an error in how I have incorporated the custom operator into the larger simulation framework but I am still figuring that out.

Some of my questions are specific to how to implement this in TOAST (and might not need a meeting of all of us). I also have some bigger picture questions that we should probably discuss as a group such as:

- How should I be quantifying the impact of pointing errors? I was thinking I could simulate a scan of a point source and then calculate some beam statistics on the resulting map. If so, what point source? What beam stats do we care about? I presume there is existing code to do some of this for LAT beams, which code should I be using?
- How specifically do we want to model pointing jitter? I am currently using a sinusoidal error with gaussian noise (specifying the noise rms) as a starting point, but we should figure out if that is how we want to do things?
- Once I can confirm that my operator is working as intended, how should I proceed with simulating CMB observations towards the ultimate goal of using that simulated data for producing 'science'.

**2026-10-12 and 2026-10-13**

Today I added the basic pointing offset operator in the same way that I tried to add the pointing jitter operator as the effects of a pointing offset operator should be easier to observe than a simple pointing jitter. Claude gave me some advice as to how to properly add the new operators and make my operator more amenable to

a generic TOAST sim. These changes include:

- Making the operator have the option to output to a new shared key instead of overwriting the default pointing. This is so that the mapmaking scripts do not use the modified pointing, but rather the nominal pointing when trying to achieve a mapmaking solution.
- Making new `apply_pointing_offset` functions that can be called from the `so_sim/simulate_data` function.

At least now I can confirm that the pointing offset operator is in fact being called, and that it is modifying the pointing as expected, as shown in the following figure.

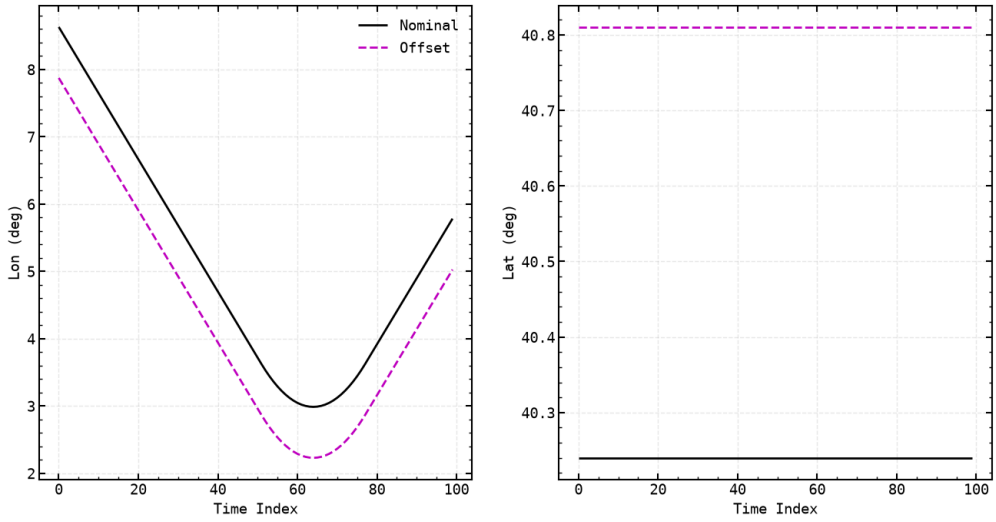



Figure 1: Pointing offset operator in action

But when I try and run the mapmaker once without offset, and once with, I still get nearly identical maps.

I am also a little concerned by the fact that my maps are not producing a sort of Airy disk pattern that I would expect from simulating a point source. I don't know if this is because of the way I am injecting a point source into my input maps, or because of how I am running the mapmaker, or something else.

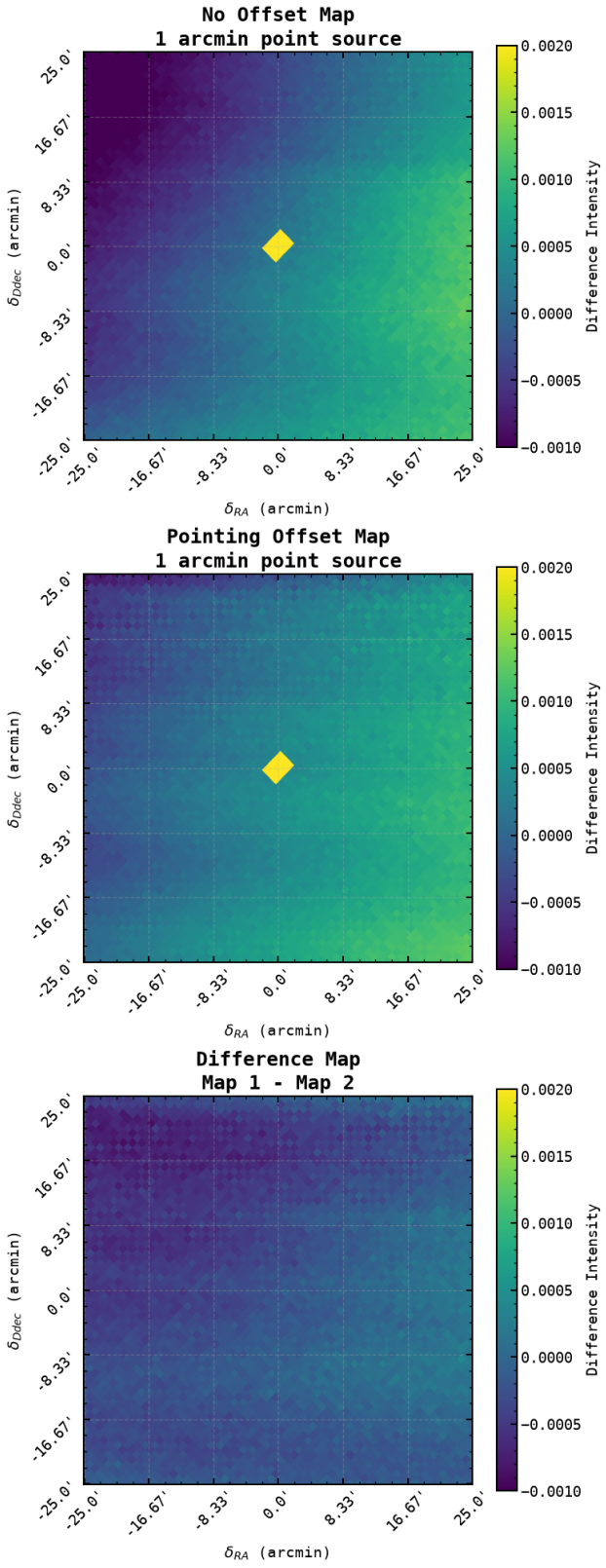



Figure 2: Pointing offset operator in action

**2026-10-11**

Adding the custom operator is unfortunately non-trivial. Just calling the operator argument with something like `--pointing_jitter.enable` does not inherently tell the sim how and when to run the operator.

Based on some trial and error (and lots of debugging print statements) it seems like code needs to be added to the so_sim.simulate_data, and wrk.pointing.py file (if not elsewhere as well).

With this in place, I reran the sims with a 2 arcmin source and pointing jitter of $\delta_{az}, \delta_{el} \sim 0.0001$ and $\delta_{az}, \delta_{el} \sim 0.001$.

Unfortunately, the output maps looks identical as shown by the following difference plot.

It is a little hard to tell if this is because the pointing jitter operator is not working, not in the right place in the code, or simply not having a level of impact strong enough to notice in this sort of map.

**2026-10-08**

I have spent the better part of the last 10 days trying to figure out how to run a more realistic LAT toast sim. Here is what I have learned:

1. A realistic SO TOAST sim is largely run through the use of the command line interface (CLI) from the sotodlib module (`toast_so_sim`). The CLI has a huge (thousands) of possible command line arguments to specify which operators to apply and operator parameters. This interface can also likely take config files to ensure reproducibility but I haven't really played around with that.

2. While adding the new pointing jitter operator to the list of possible operators to call in the CLI is fairly straightforward, it is not clear how/where to include calling the new operator in the pipeline of a toast sim.

3. The large number of arguments are not really accompanied by clear instructions indicating which operators need to be specified for a particular type of simulation. For instance, I have been trying to scan a solar system object (SSO) for a semi-realistic simulation of a point-source. Turns out defining the SSO in your schedule does not make toast actually simulate the source, to do this, you need to enable the `--sim_sso.enable` argument. But this argument seems to also require other arguments to be specified. What those arguments are, and what they should be set to is not clear (for example, what beam file should I specify).

4. Running from a CLI makes many interactive things like plotting the impact of my operator much more difficult.

Before I can do anything, I need to make sure that running the toast sim CLI actually pull from my custom code (since my code is not yet incorporated into the greater TOAST, sotodlib packages). The instructions for doing this are mostly documented on the SO wiki page for using NERSC. In brief, the steps are:

1. Create a custom conda environment.

2. Install the sotodlib and toast packages from source.

Once this is done, the new operator must be added to the list of available operators for `toast_so_sim`

Because I did not want to deal with trying to figure out how to use the toast SSO method, I just had toast spit out the RA/Dec of where Mars was from its schedule, then adjusted my input maps to have a point source at that location.

To test that the CLI method was working, I ran a toast sim of my fake 'Mars' map without any pointing jitter. It took me a minute to get the inputs correct so that it would actually scan my custom map, and also run a filter bin mapmaker on the resulting TOD. After some trial and error, I was able to get the following map.

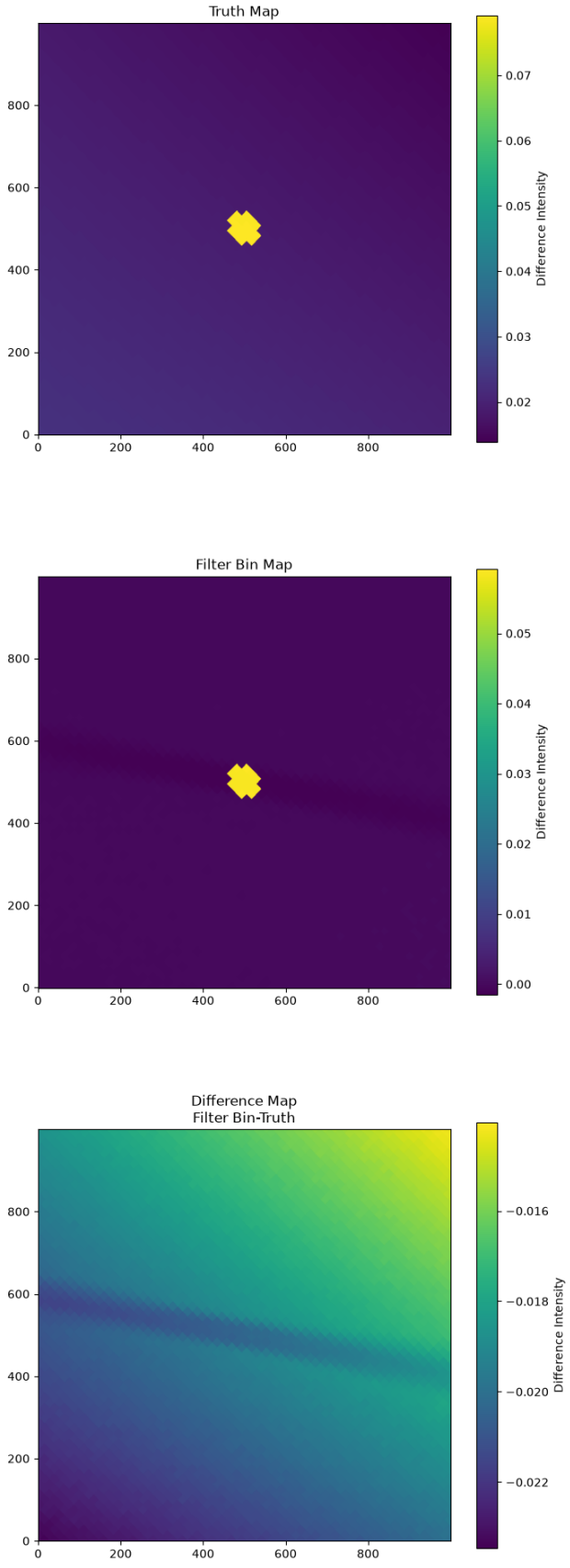



Figure 3: Difference between input map and filter bin scan.

**2026-09-30**

I have a simple simulation pipeline that can make a toast simulation scanning a custom map, with or without pointing jitter. Then run a ml mapmaker or a filter bin mapmaker on the resulting TOD.

The input map looks like this - for a 10arcmin source:

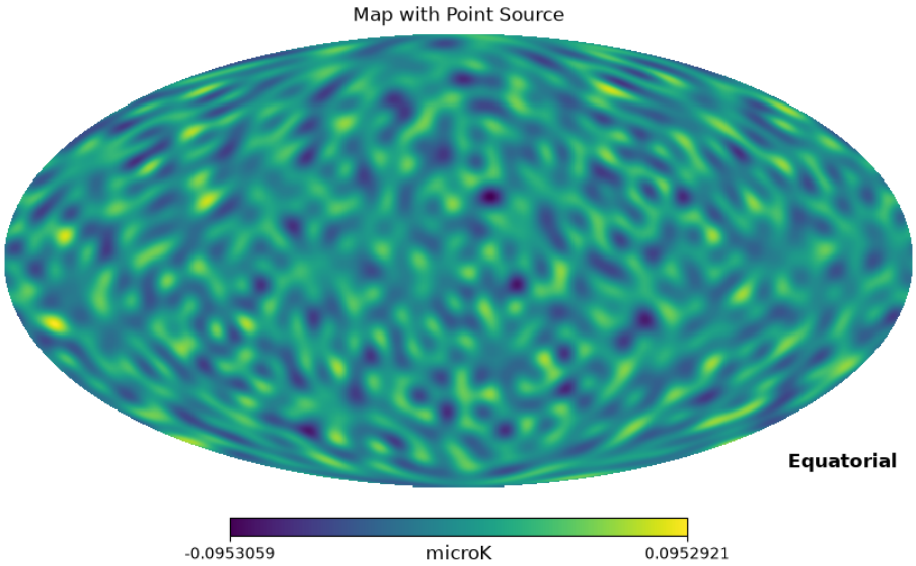



Figure 4: Input map for the simulation.

The toast sim also produces some pointing visualizations to confirm that the pointing jitter is being applied as expected.

This can be viewed in 3D:

After the ml mapmaker, the resulting map look like:

This work has been a good proof of concept for implementing a pointing jitter, however, there are major limitations to this setup for simulating a realistic LAT scan. First, this telescope/focal plane objects are not representative of a realistic LAT. Second, other elements of the simulation like noise, atmosphere, etc. are not included in a realistic manner. It would see that the best way to proceed is to learn how the realistic LAT sims are actually run and incorporate my pointing jitter into that framework.

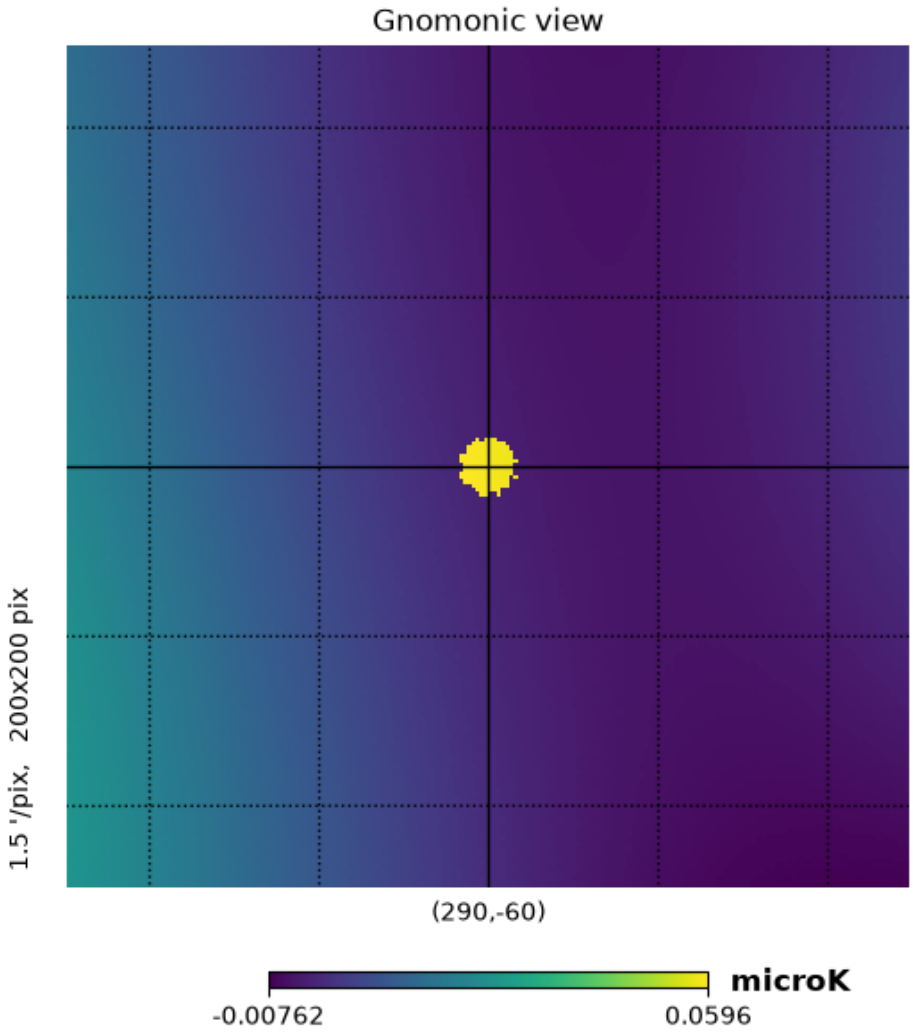



Figure 5: Input map for the simulation - gnomonic projection of the source.

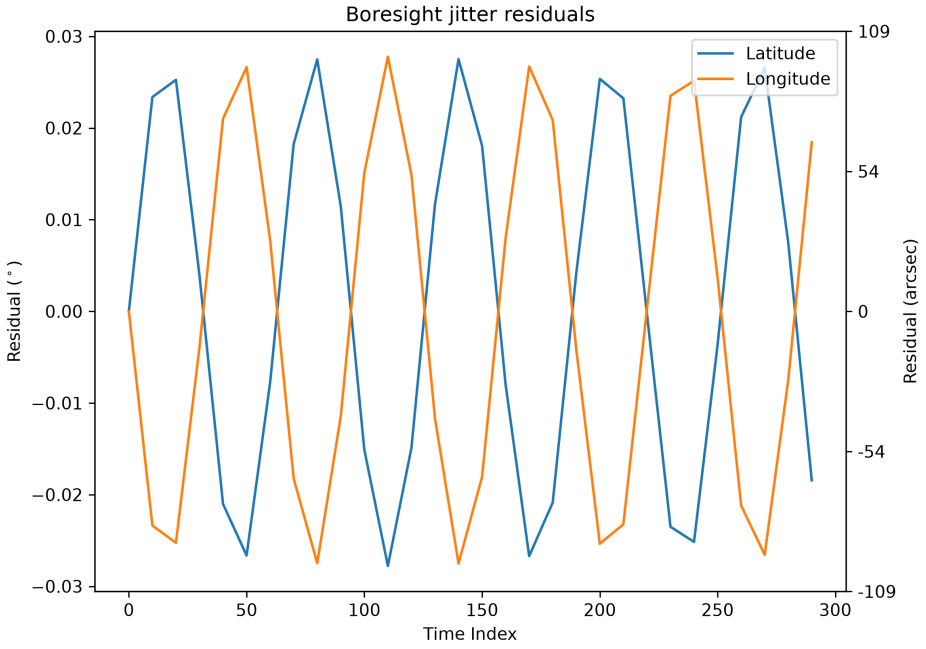



Figure 6: Pointing jitter boresight residuals.

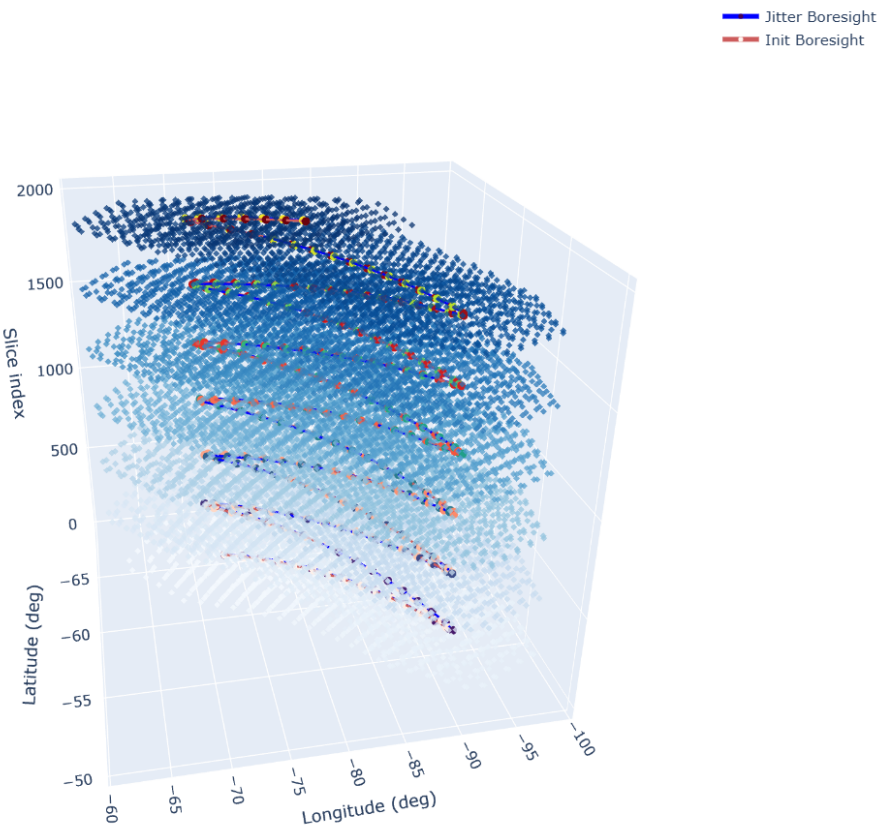



Figure 7: Pointing jitter boresight residuals.

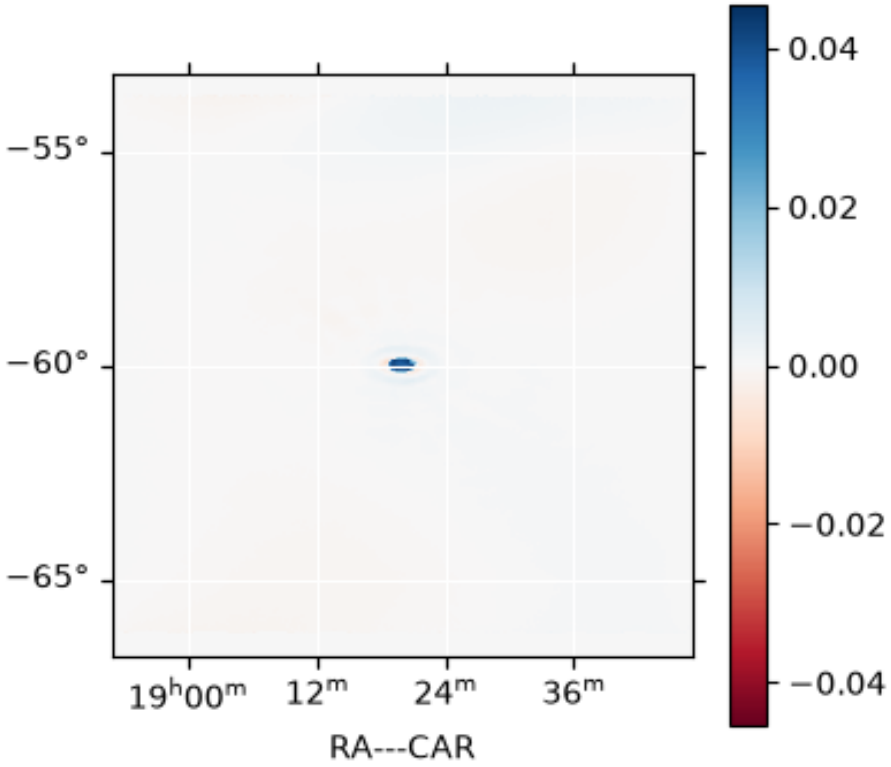



Figure 8: Output I map of ML mapmaker with no jitter.